

一、测试结果

1.1 运算符与操作数检查

```
wmh0121@localhost:~/ics-lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 12/24 operators
midpointTowardFirst: 17/32 operators
isBetweenEitherOrder: 29/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 21/52 operators
floatScaleThreeHalves: 31/60 operators
floatRoundEven: 27/65 operators
float_i2f: 35/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 1 运算符与操作数检查结果

1.2 正确性测试

```
wmh0121@localhost:~/ics-lab1$ ./btest
Score   Rating  Errors  Function
1       1       0       signMask
2       2       0       bitXor
3       3       0       negativePart
4       4       0       copyByteWithin
4       4       0       logicalShift
4       4       0       swapNibblePairs
4       4       0       secondLowestZeroBit
5       5       0       oddParity
5       5       0       rotateRightBits
5       5       0       roundEvenPow2
5       5       0       midpointTowardFirst
7       7       0       isBetweenEitherOrder
7       7       0       mul5Sat
7       7       0       classifyAdd3
7       7       0       floatScaleThreeHalves
10      10      0       floatRoundEven
10      10      0       float_i2f
10      10      0       bitCount
10      10      0       bitReverse
Total points: 110/110
wmh0121@localhost:~/ics-lab1$ |
```

图 2 btest 正确性测试结果

二、各函数实现思路

P1 signMask

题目要求返回只有最高位为 1、其余位全部为 0 的 32 位掩码。32 位整数的位编号从 0 开始，因此最高位是第 31 位。常量 1 的二进制表示只有最低位为 1，把它左移 31 位后，这个 1 就会移动到最高位，得到位模式 0x80000000。因此直接返回 $1 \ll 31$ 就行。

P2 bitXor

异或的规则是两个对应位不同时结果为 1，相同时结果为 0，但本题只能使用按位取反 $\sim$ 和按位与 $\&$ 。两个位相同有两种情况：两个位都为 1，可以用 $x \& y$ 找出；两个位都为 0，可以先分别取反，再用 $\sim x \& \sim y$ 找出。

因此， $\sim(x \& y)$ 表示排除“同时为 1”的位置， $\sim(\sim x \& \sim y)$ 表示排除“同时为 0”的位置。把这两个结果按位与，只有两个输入位不同的位置会保留为 1，所以最终表达式为 $\sim(x \& y) \& \sim(\sim x \& \sim y)$ 。

P3 negativePart

题目要求当 x 为负数时返回 $-x$ ，当 x 为非负数时返回 0。对有符号整数执行算术右移时，负数高位补 1，非负数高位补 0。因此， $x \gg 31$ 可以得到符号掩码：负数对应全 1，非负数对应全 0。

补码取负可以写成 $\sim x + 1$ 。将它与符号掩码按位与，负数会保留取负结果，非负数会被全 0 掩码清零。对于 `INT_MIN`，按照实验规定的 32 位位向量模型，取负后仍保留相同的低 32 位位模式。

P4 copyByteWithin

题目要求把 x 的第 src 个字节复制到第 dst 个字节，同时保持其他字节不变。一个字节占 8 位，所以第 src 个字节的起始位置是 $src \times 8$ 。由于不能使用乘法，代码用 $src \ll 3$ 和 $dst \ll 3$ 分别计算源字节和目标字节的移位置。

先将 x 右移源字节对应的位数，再与 255 按位与，取出源字节。接着把 255 左移到目标字节位置，生成目标字节掩码；将掩码取反后与 x 相与，可以清除目标字节而保留其他字节。最后把源字节移到目标位置并按位或回去。若 src 与 dst 相同，清除后写回的仍是原字节，结果保持不变。

P5 logicalShift

逻辑右移要求高位补 0，但 C 语言对有符号整数执行右移时采用算术右移，负数高位会补 1。代码先计算 $x \gg n$ ，再构造一个高 n 位为 0、其余位为 1 的掩码，清除算术右移补入的符号位。

$1 \ll 31$ 先生成只有最高位为 1 的数。对它算术右移 n 位、再左移 1 位并取反，得到 $\sim(((1 \ll 31) \gg n) \ll 1)$ 。将右移结果与该掩码按位与，就得到逻辑右移。 $n = 0$ 时掩码为全 1，原数不变； $n = 31$ 时掩码只保留最低位。

P6 swapNibblePairs

题目要求交换每个字节内部的高 4 位和低 4 位。代码先从常量 15 构造掩码 0x0F0F0F0F： $15 | (15 \ll 8)$ 得到低 16 位中的两个 0x0F，再把该结果左移 16 位并按位或，扩展到完整 32 位。

$x \& \text{mask}$ 取出每个字节的低 4 位，左移 4 位后移动到本字节高 4 位； $(x \gg 4) \& \text{mask}$ 取出原来的高 4 位并放入低 4 位。将两部分按位或，即可完成四个字节内部的半字节交换。

P7 secondLowestZeroBit

题目要求返回一个掩码，标记 x 中从低位向高位第二个 0 所在的位置。对一个二进制数执行 $x + 1$ 时，进位会传播到最低的 0 位，使这个 0 变成 1，并把它右侧连续的 1 变成 0。因此， $x | (x + 1)$ 会把最低的 0 填成 1，同时保留更高位置的位。

将这个结果记为 filled ，此时 filled 中最低的 0 正好是原数第二低的 0。表达式 $\sim \text{filled} \& (\text{filled} + 1)$ 可以隔离出这个位置。如果原数中不足两个 0，例如 $x = -1$ ，计算结果自然为 0。

P8 oddParity

题目要求返回奇校验位：如果 x 的二进制表示中 1 的个数为偶数，返回 1；如果为奇数，返回 0。异或可以保留一组比特中 1 的个数的奇偶性，因此采用分治折叠。

代码依次把 x 与右移 16、8、4、2、1 位后的结果异或，不断把高半部分的信息折叠到低半部分。最后最低位表示原来 32 位中 1 的个数是奇数还是偶数。最低位为 0 表示 1 的个数为偶数，所以对 $x \& 1$ 取逻辑非即可得到题目要求的结果。

P9 rotateRightBits

循环右移与普通右移不同：从右端移出的位需要重新放到左端。题目允许 n 大于 31，因此先使用 $n = n \& 31$ ，把旋转位数化为对 32 取模后的结果。

右侧部分按照 P5 的方法完成逻辑右移；左侧部分把原来右端被移出的位移动到高位。为了避免 $n = 0$ 时出现左移 32 位，左移量写成 $(\sim n + 1) \& 31$ ，等价于 $(-n) \bmod 32$ 。最后把逻辑右移部分和左移部分按位或，得到循环右移结果。

P10 roundEvenPow2

题目要求把非负整数 x 舍入到最接近的 2^n 的倍数；如果恰好位于两个倍数中间，则选择除以 2^n 后商为偶数的结果。先用 $\text{block} = 1 \ll n$ 表示一个舍入区间的大小，并令偏置 bias 等于半个区间减 1。

$(x \gg n) \& 1$ 可以取得当前商的最低位，判断商是奇数还是偶数。计算 $x + \text{bias} + \text{odd}$ 时，余数小于半程不会进入下一区间，余数大于半程会向上进入下一区间；恰好等于半程时，只有当前商为奇数才额外加 1，从而选择偶数商。最后用 $\sim(\text{block} - 1)$ 清除最低 n 位，得到最终的 2^n 倍数。

P11 midpointTowardFirst

题目要求计算 x 与 y 的数学中点，同时避免直接计算 $x + y$ 造成溢出。如果中点位于两个整数之间，还要选择更靠近第一个参数 x 的整数。表达式 $(x \& y) + ((x \wedge y) \gg 1)$ 可以在不直接相加的情况下求出中点的向下取整值。

当 $(x \wedge y) \& 1$ 为 1 时，两数之和为奇数，中点恰好落在两个整数之间。此时如果 $x > y$ ，就在向下取整值上加 1；如果 $x < y$ ，保持向下取整值。比较大小时先区分两数是否异号：异号时根据符号判断，同号时根据 $y - x$ 的符号判断，从而避免比较过程因溢出而出错。

P12 isBetweenEitherOrder

题目要求判断 x 是否位于端点 a 和 b 构成的闭区间中, 而且端点输入顺序不固定。代码分别判断 $x < a$ 和 $x < b$ 。如果两个判断的结果不同, 说明 x 严格位于两个端点之间。

直接用差值判断大小可能在异号情况下溢出, 因此先比较符号。若 x 与端点异号, 负数一定小于非负数; 若二者同号, 再通过 $x - a$ 或 $x - b$ 的符号判断。最后加入 $x == a$ 和 $x == b$ 两种情况, 使判断范围包含两个端点。

P13 mul5Sat

题目要求计算 $x \times 5$; 如果精确结果超过 INT_MAX , 返回 INT_MAX , 如果低于 INT_MIN , 返回 INT_MIN 。代码没有使用乘法, 而是依次计算 $2x$ 、 $4x$ 和 $5x$: 先用 $x + x$ 得到 $twice$, 再用 $twice + twice$ 得到 $fourTimes$, 最后加上 x 得到 $fiveTimes$ 。

补码加法溢出时, 结果符号可能与原数不同。代码分别计算 $x \wedge twice$ 、 $x \wedge fourTimes$ 和 $x \wedge fiveTimes$, 只要任一步的最高位发生变化, 就把溢出掩码设为全 1。这样即使后续回绕使最终符号再次变化, 也不会漏掉前面已经发生的溢出。根据原始 x 的符号构造 INT_MAX 或 INT_MIN , 最后用溢出掩码选择饱和值或正常的 $fiveTimes$ 。

P14 classifyAdd3

题目要求判断三个 32 位整数的精确数学和 $x + y + z$ 是否超出 int 范围, 而且不能使用更宽的整数类型。直接计算三数之和可能在中间步骤溢出, 因此先把每个数写成 $4q + r$ 。其中 q 由算术右移 2 位得到, r 是最低两位, 范围为 0 到 3。

代码先相加三个商, 再加入三个余数相加产生的进位, 得到 $high = floor((x + y + z) / 4)$ 。输入缩小四倍后, 中间结果仍在 32 位范围内。若 $high \geq 2^{29}$, 精确总和至少为 2^{31} , 说明超过 INT_MAX , 返回 1; 若 $high < -2^{29}$, 精确总和低于 INT_MIN , 返回 -1; 其他情况返回 0。两个越界条件互斥, 按位或后即可得到最终分类。

P15 floatScaleThreeHalves

题目要求根据单精度浮点数的位模式计算 $f \times 3/2$, 并采用舍入到最近偶数的规则。先从 uf 中拆出符号位、8 位阶码和 23 位尾数字段。阶码为 255 时, 输入是 NaN 或无穷大, 直接返回原位模式。规格化数需要补上隐含的最高有效位 $0x800000$, 非规格化数直接使用原尾数。

乘以 3 通过 $significand + (significand \ll 1)$ 完成, 再根据乘积的最高位决定右移 1 位还是 2 位。若需要右移 2 位, 同时把阶码加 1, 使结果重新规格化。右移时记录被舍弃的余数: 余数大于半程时进位, 等于半程且保留结果最低位为 1 时也进位, 从而实现最近偶数舍入。非规格化数可能在舍入后进入规格化范围; 阶码达到 255 时返回相同符号的无穷大。符号位始终保留, 因此正负零也能保持原符号。

P16 floatRoundEven

题目要求把单精度浮点数舍入到最近整数, 但返回值仍是该整数对应的单精度浮点位模式。恰好位于两个整数中间时选择偶数整数, 舍入到零时保留输入符号。代码先根据阶码判断数值范围。阶码至少为 150 时, 无偏阶码至少为 23, 编码已经没有需要舍弃的小数位, NaN 和无穷大也包含在这一情况中, 因此直接返回原值。

阶码小于 126 时, 绝对值小于 0.5, 返回带原符号的零。阶码等于 126 时, 尾数为 0 表示恰好是 0.5, 按偶数规则舍入到 0; 尾数非零表示绝对值大于 0.5, 舍入到带相同符号的 1.0。其余情况用 $shift = 150 - exponent$ 计算需要清除的小数位, 比较被清除部

分与半程值。大于半程时进位；等于半程且当前整数为奇数时进位，最终实现最近偶数舍入。

P17 float_i2f

题目要求把一个 32 位 int 转换为对应的单精度浮点模式。 $x = 0$ 时直接返回浮点正零。对于负数，先保存符号位，再用补码取负得到无符号绝对值。使用 unsigned 保存绝对值，可以正确处理绝对值无法放入正 int 的 INT_MIN。

代码从第 31 位开始向下查找最高的 1，最高位位置加 127 得到浮点阶码。如果有效位不超过 24 位，只需左移对齐，转换是精确的；如果有效位超过 24 位，就右移并记录被舍弃的余数。余数大于半程时进位，等于半程且保留有效数最低位为 1 时也进位。若进位使有效数产生新的最高位，就再右移一位并增加阶码。最后组合符号位、阶码和低 23 位尾数。

P18 bitCount

题目要求统计 32 位整数中 1 的总个数。整数题不允许使用循环，因此采用分治并行计数。代码从小常量 85、51 和 15 构造 0x55555555、0x33333333 和 0x0F0F0F0F 三种掩码。

第一步用 0x55555555 把每相邻两位中的两个比特相加，使每个 2 位组保存其中 1 的数量；第二步用 0x33333333 合并相邻的 2 位组；第三步用 0x0F0F0F0F 合并相邻的 4 位组。随后通过右移 8 位和 16 位继续相加，把四个字节的计数汇总到最低部分。32 位中最多有 32 个 1，最低 6 位足以保存结果，所以最后与 63 相与得到最终计数。

P19 bitReverse

题目要求把 32 位整数的所有比特完全反转。代码采用分治交换，逐步交换越来越大的相邻分组。先用 $255 | (255 \ll 8)$ 构造 0x0000FFFF，再通过当前掩码与其左移结果异或，依次得到 0x00FF00FF、0x0F0F0F0F、0x33333333 和 0x55555555。

交换从最小分组开始：先交换相邻单个位，再交换相邻 2 位组、4 位组、字节，最后交换高低两个 16 位半字。每一步都用掩码取出两部分，分别向左或向右移动后按位或合并。右移产生的符号扩展会被掩码清除。经过 1、2、4、8、16 位五个层次的交换后，第 0 位移动到第 31 位，第 1 位移动到第 30 位，最终完成全部 32 位的反转。